BÁO CÁO KẾ HOẠCH TRIỂN KHAI ĐỀ TÀI

BÃI ĐỖ XE THÔNG MINH LIÊN KẾT NHIỀU BÃI

Môn học: Hệ thống phân tán
Lĩnh vực: Vận hành / IoT / Logistics

---

1. GIỚI THIỆU ĐỀ TÀI

1.1. Tên đề tài

Bãi đỗ xe thông minh liên kết nhiều bãi

1.2. Bối cảnh

Trong thực tế, mỗi bãi đỗ xe thường hoạt động tương đối độc lập. Thông tin về số lượng vị trí còn trống của từng bãi được quản lý riêng, khiến người dùng phải đến trực tiếp hoặc tìm kiếm từng bãi để biết còn chỗ hay không.

Khi số lượng bãi đỗ tăng lên, việc xây dựng một hệ thống tập trung duy nhất để quản lý toàn bộ dữ liệu có thể làm giảm tính độc lập của từng bãi và tạo ra một điểm lỗi tập trung.

Do đó, đề tài xây dựng hệ thống trong đó:

- Mỗi bãi đỗ xe là một **node độc lập**.
- Mỗi node tự quản lý trạng thái vị trí đỗ xe của mình.
- Dữ liệu của các bãi được lưu trữ riêng biệt.
- Các node giao tiếp với nhau thông qua mạng.
- Một dịch vụ trung gian tổng hợp thông tin từ nhiều bãi.
- Người dùng chỉ cần sử dụng một ứng dụng duy nhất để tìm kiếm và đặt chỗ.

Cách tổ chức này phù hợp với khái niệm hệ thống phân tán trong tài liệu môn học: các thành phần được đặt trên các máy tính mạng, trao đổi thông tin và phối hợp hoạt động thông qua truyền tin nhưng đối với người sử dụng vẫn tạo cảm giác như một hệ thống duy nhất.

---

2. MỤC TIÊU ĐỀ TÀI

2.1. Mục tiêu tổng quát

Xây dựng một hệ thống quản lý nhiều bãi đỗ xe độc lập theo kiến trúc phân tán, cho phép người dùng:

- Xem danh sách nhiều bãi đỗ xe.

- Xem số lượng chỗ còn trống.
- Tìm kiếm bãi còn chỗ.
- Xem trạng thái từng vị trí đỗ xe.
- Đặt trước vị trí.
- Hủy đặt chỗ.
- Theo dõi trạng thái bãi gần thời gian thực.

Mỗi bãi đỗ xe phải có khả năng hoạt động tương đối độc lập và dữ liệu của bãi đó phải được quản lý tại node tương ứng.

2.2. Mục tiêu về hệ thống phân tán

Hệ thống phải chứng minh được ba yếu tố chính:

Thứ nhất: Có nhiều thành phần chạy độc lập

Tối thiểu triển khai:

- Aggregator Service.
- Parking Node A.
- Parking Node B.

Có thể triển khai thêm:

- Parking Node C.
- Frontend.
- Authentication Service.

Các thành phần phải chạy dưới các process/container độc lập và giao tiếp thông qua mạng.

Không được chỉ chia code thành nhiều thư mục hoặc nhiều class nhưng tất cả chạy trong cùng một process.

Thứ hai: Dữ liệu được phân tán thực sự

Không sử dụng một database duy nhất chứa toàn bộ trạng thái của tất cả các bãi.

Ví dụ:

Parking A Database

```
Parking A
├── Slot A01
├── Slot A02
├── Slot A03
└── Reservation A...
```

Parking B Database

```
Parking B
├── Slot B01
├── Slot B02
├── Slot B03
└── Reservation B...
```

Node A không trực tiếp truy cập database của Node B và ngược lại.

Muốn lấy dữ liệu của Node B, các thành phần khác phải gửi request qua mạng.

Tài liệu môn học cũng nhấn mạnh các máy của hệ thống phân tán không chia sẻ bộ nhớ và việc trao đổi thông tin được thực hiện thông qua cơ chế truyền tin.

Thứ ba: Có nghiệp vụ cụ thể

Hệ thống không chỉ thực hiện CRUD dữ liệu.

Các nghiệp vụ chính gồm:

1. Tra cứu chỗ trống trên nhiều bãi.
2. Đặt vị trí đỗ xe.
3. Xe vào bãi.
4. Xe rời bãi.
5. Hủy đặt chỗ.
6. Cập nhật trạng thái chỗ trống.
7. Xử lý trường hợp một node không phản hồi.

---

3. PHẠM VI HỆ THỐNG

Phiên bản đầu tiên dự kiến triển khai **3 bãi đỗ xe**:

Node	Tên	Số slot mô phỏng
Node A	Parking A	20
Node B	Parking B	30
Node C	Parking C	15

Mỗi bãi hoạt động như một hệ thống độc lập.

Trong quá trình demo có thể chạy tất cả trên cùng một máy tính nhưng phải thông qua:

- Port khác nhau.
- Process/container khác nhau.
- Database khác nhau.

Ví dụ:

```
Aggregator     localhost:8000
Parking A      localhost:8001
Parking B      localhost:8002
Parking C      localhost:8003
Frontend       localhost:3000
```

Việc chạy trên cùng một máy chỉ phục vụ mục đích demo. Về mặt kiến trúc, các node vẫn có khả năng triển khai trên những máy khác nhau.

---

4. ĐỐI TƯỢNG SỬ DỤNG

4.1. Người dùng

Người cần tìm vị trí đỗ xe.

Chức năng:

- Đăng nhập.
- Xem danh sách bãi.
- Kiểm tra chỗ trống.
- Xem chi tiết bãi.
- Đặt chỗ.
- Hủy đặt.
- Xem lịch sử đặt chỗ.

4.2. Nhân viên bãi xe

Quản lý hoạt động tại một bãi cụ thể.

Chức năng:

- Xác nhận xe vào.
- Xác nhận xe ra.
- Xem trạng thái slot.
- Khóa slot khi bảo trì.
- Xem danh sách reservation.

4.3. Quản trị viên

Quản lý hệ thống tổng thể.

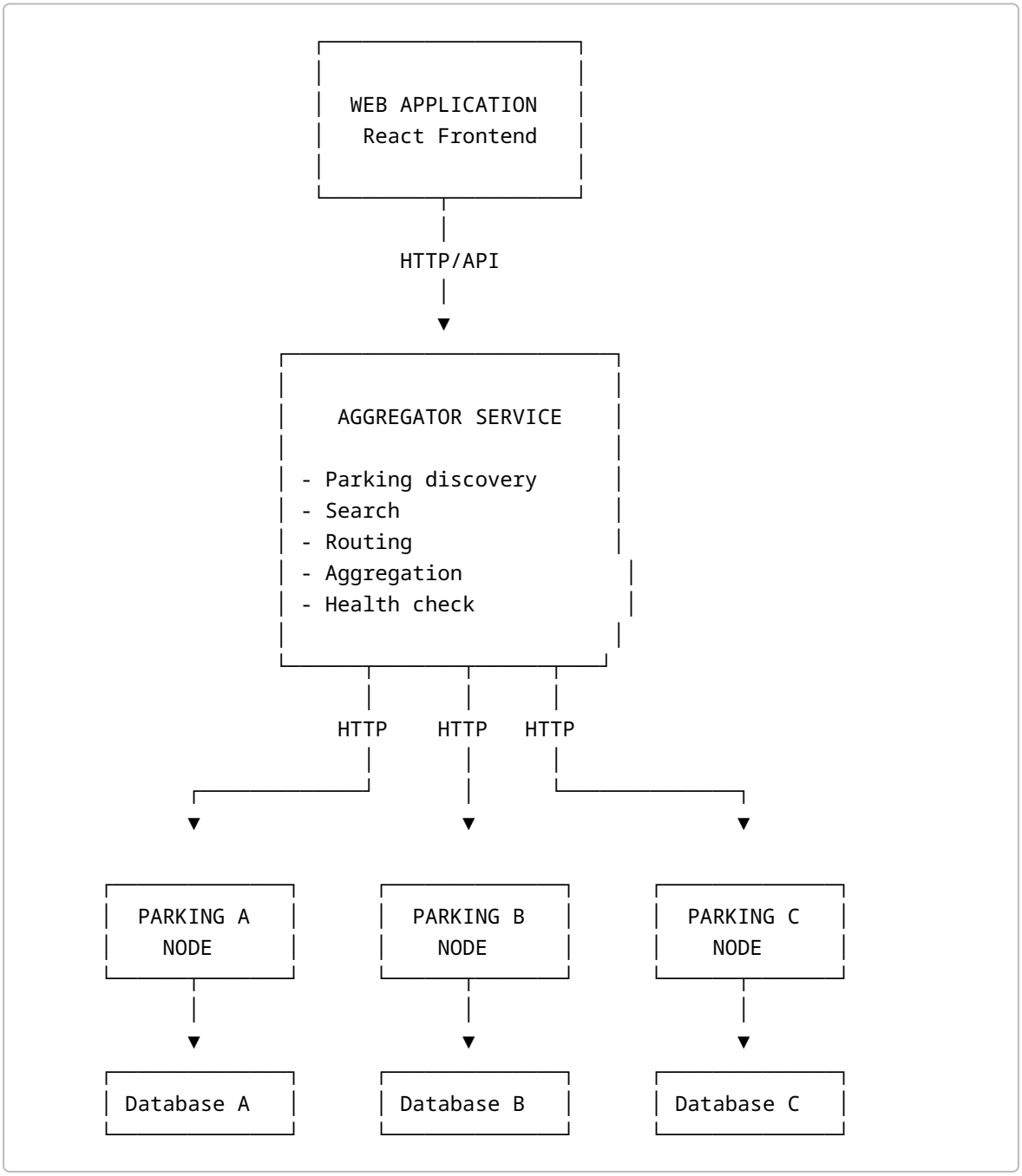
Chức năng:

- Quản lý thông tin các Parking Node.
- Kiểm tra trạng thái online/offline.
- Xem thống kê.
- Theo dõi lỗi kết nối giữa các node.

---

5. KIẾN TRÚC TỔNG THỂ

Kiến trúc đề xuất:



6. GIẢI THÍCH CÁC THÀNH PHẦN

6.1. Frontend

Frontend là ứng dụng giao tiếp với người dùng.

Đề xuất:

ReactJS

Frontend không được truy cập trực tiếp database của các bãi.

Luồng:

```
Frontend
   ↓
Aggregator
   ↓
Parking Node
```

---

7. AGGREGATOR SERVICE

Aggregator là thành phần quan trọng nhất trong việc liên kết nhiều bãi.

Nhiệm vụ:

- Lưu danh sách các Parking Node.
- Kiểm tra node đang online hay offline.
- Nhận request từ frontend.
- Gửi request đến các node.
- Tổng hợp kết quả.
- Trả kết quả thống nhất cho frontend.

Ví dụ người dùng yêu cầu:

```
GET /api/parkings/availability
```

Aggregator gửi đồng thời:

```
GET Parking A /api/availability

GET Parking B /api/availability
```

```
GET Parking C /api/availability
```

Sau đó tổng hợp kết quả:

```
[
  {
    "parkingId": "A",
    "available": 12,
    "total": 20,
    "status": "ONLINE"
  },
  {
    "parkingId": "B",
    "available": 7,
    "total": 30,
    "status": "ONLINE"
  },
  {
    "parkingId": "C",
    "available": 0,
    "total": 15,
    "status": "ONLINE"
  }
]
```

Đây chính là chức năng **tra cứu tổng hợp đồng thời nhiều node** theo yêu cầu đề tài.

---

8. PARKING NODE

Mỗi Parking Node đại diện cho một bãi xe.

Ví dụ:

```
Parking Node A
```

chỉ chịu trách nhiệm quản lý:

```
Parking A
```

Node A không quản lý Parking B hoặc Parking C.

Parking Node thực hiện:

- Quản lý slot.

- Nhận sự kiện xe vào.
- Nhận sự kiện xe ra.
- Xử lý reservation.
- Kiểm tra slot trống.
- Cập nhật trạng thái.
- Trả dữ liệu qua API.

---

9. PHÂN TÁN DỮ LIỆU

Đây là phần quan trọng nhất để chứng minh đề tài không phải CRUD thông thường.

9.1. Nguyên tắc

Dữ liệu được phân đoạn theo **parking_id**.

```
Parking A → Database A
Parking B → Database B
Parking C → Database C
```

9.2. Lý do phân đoạn

Mỗi bãi là một đơn vị vận hành độc lập.

Nếu Parking B mất kết nối:

```
Parking A → vẫn hoạt động
Parking C → vẫn hoạt động
Parking B → OFFLINE
```

Hệ thống không bị ngừng hoàn toàn.

Việc đặt dữ liệu tại nhiều nơi cũng phù hợp với mô tả hệ thống thông tin phân tán trong tài liệu: dữ liệu có thể được đặt ở nhiều nơi trong khi quá trình xử lý giữa các nơi có liên quan với nhau.

---

10. THIẾT KẾ DATABASE

10.1. Database tại mỗi Parking Node

Mỗi node sử dụng database riêng nhưng có thể dùng cùng schema.

Bảng `parking_slots`

Field	Type	Ý nghĩa
id	UUID/INT	ID
slot_code	VARCHAR	A01, A02...
floor	INT	Tầng
type	VARCHAR	CAR/MOTORBIKE
status	VARCHAR	AVAILABLE/OCCUPIED/RESERVED/MAINTENANCE
updated_at	DATETIME	Thời gian cập nhật

Bảng `reservations`

Field	Type
id	UUID
user_id	UUID
slot_id	UUID
license_plate	VARCHAR
start_time	DATETIME
expire_time	DATETIME
status	VARCHAR
created_at	DATETIME

Bảng `parking_events`

Lưu lịch sử trạng thái.

Field	Ý nghĩa
id	ID sự kiện
slot_id	Slot
event_type	CAR_ENTER / CAR_EXIT / RESERVED
timestamp	Thời gian
license_plate	Biển số

---

11. DATABASE CỦA AGGREGATOR

Aggregator không lưu trạng thái chi tiết từng slot.

Chỉ cần lưu metadata của Parking Node.

`parking_nodes`

Field	Ví dụ
parking_id	A
name	Parking PTIT
api_url	http://parking-a:8001
address	Hà Nội
latitude	...
longitude	...
status	ONLINE
last_seen	...

Điều này đảm bảo dữ liệu slot vẫn thuộc quyền quản lý của từng node.

---

12. THIẾT KẾ API

12.1. Parking Node API

Kiểm tra trạng thái

```
GET /health
```

Response:

```
{
  "status": "UP",
  "parkingId": "A"
}
```

Lấy số chỗ trống

```
GET /api/availability
```

Lấy danh sách slot

```
GET /api/slots
```

Lấy slot trống

```
GET /api/slots/available
```

Đặt slot

```
POST /api/reservations
```

Request:

```
{
  "userId": "USER01",
  "slotId": "A01",
  "licensePlate": "30A-12345"
}
```

Xe vào

```
POST /api/slots/{slotId}/enter
```

Xe ra

```
POST /api/slots/{slotId}/exit
```

Hủy reservation

```
DELETE /api/reservations/{id}
```

---

13. AGGREGATOR API

Frontend chỉ cần làm việc với Aggregator.

Danh sách bãi

```
GET /api/parkings
```

Tra cứu toàn hệ thống

```
GET /api/parkings/availability
```

Tìm bãi còn chỗ

```
GET /api/parkings/search?available=true
```

Chi tiết bãi

```
GET /api/parkings/{parkingId}
```

Đặt chỗ

```
POST /api/parkings/{parkingId}/reservations
```

Aggregator sẽ route request đến node tương ứng.

---

14. NGHIỆP VỤ 1 – TRA CỨU CHỖ TRỐNG

Luồng xử lý:

```
User
 ↓
Frontend
 ↓
Aggregator
```

```
 ↓
 ├── Parking A
 ├── Parking B
 └── Parking C
 ↓
Aggregator tổng hợp
 ↓
Frontend
 ↓
User
```

Aggregator nên gửi request tới các Parking Node **song song**, không gửi tuần tự.

Ví dụ:

```
A ──────► 12 chỗ
B ──────►  7 chỗ
C ──────►  2 chỗ
```

Aggregator trả:

```
A: 12
B: 7
C: 2
```

---

15. NGHIỆP VỤ 2 – ĐẶT CHỖ

Ví dụ người dùng đặt tại Parking B:

```
User
 ↓
Aggregator
 ↓
Parking B
 ↓
DB B
```

Parking B thực hiện:

1. Kiểm tra slot.
2. Kiểm tra slot có AVAILABLE không.
3. Nếu AVAILABLE thì chuyển sang RESERVED.
4. Tạo reservation.
5. Trả kết quả.

Khi có nhiều người cùng đặt một slot, Parking Node phải đảm bảo chỉ một request thành công.

---

16. NGHIỆP VỤ 3 – XE VÀO BÃI

Ví dụ:

```
A05 = RESERVED
```

Xe đến:

```
POST /slots/A05/enter
```

Node cập nhật:

```
RESERVED
   ↓
OCCUPIED
```

Đồng thời số chỗ trống của bãi thay đổi.

---

17. NGHIỆP VỤ 4 – XE RA

Xe rời bãi:

```
POST /slots/A05/exit
```

Trạng thái:

```
OCCUPIED
   ↓
AVAILABLE
```

Số chỗ trống được tăng lên.

---

18. CẬP NHẬT GẦN THỜI GIAN THỰC

Theo yêu cầu đề tài, trạng thái chỗ trống cần cập nhật gần thời gian thực khi xe vào hoặc ra.

Phiên bản triển khai đề xuất:

```
Parking Node
      ↓
Event
      ↓
Aggregator
      ↓
WebSocket
      ↓
Frontend
```

Khi Parking A thay đổi:

```
{
  "event": "SLOT_UPDATED",
  "parkingId": "A",
  "slot": "A05",
  "status": "OCCUPIED"
}
```

Aggregator nhận sự kiện và gửi xuống các client đang kết nối.

Mô hình dựa trên sự kiện cũng xuất hiện trong tài liệu môn học, trong đó các tiến trình trao đổi thông tin thông qua việc phát tán sự kiện và các tiến trình đăng ký sẽ nhận sự kiện tương ứng.

---

19. XỬ LÝ NODE BỊ LỖI

Đây là tình huống nên demo để thể hiện tính phân tán.

Giả sử:

```
Parking B OFFLINE
```

Aggregator gọi:

```
A → OK
B → Timeout
C → OK
```

Kết quả:

```
[
  {
    "parking": "A",
    "status": "ONLINE",
    "available": 12
  },
  {
    "parking": "B",
    "status": "OFFLINE"
  },
  {
    "parking": "C",
    "status": "ONLINE",
    "available": 4
  }
]
```

Hệ thống vẫn tiếp tục hoạt động với A và C.

---

20. HEALTH CHECK

Aggregator định kỳ:

```
GET /health
```

đến từng node.

Ví dụ:

```
Parking A → UP
Parking B → UP
Parking C → DOWN
```

Có thể health check sau mỗi:

```
5-10 giây
```

---

21. TÍNH TRONG SUỐT

Người dùng không cần biết dữ liệu Parking A nằm ở đâu hoặc Parking B chạy trên máy nào.

Họ chỉ truy cập:

```
Smart Parking Application
```

Đây là đặc tính trong suốt của hệ thống phân tán.

Tài liệu mô tả tính trong suốt nhằm che giấu vị trí thực của thông tin và nơi xử lý đối với người sử dụng.

---

22. TÍNH MỞ RỘNG

Ban đầu:

```
A
B
C
```

Sau này có thể thêm:

```
D
E
F
```

mà không cần thay đổi kiến trúc tổng thể.

Chỉ cần đăng ký node mới với Aggregator.

Tài liệu môn học xác định khả năng bổ sung máy tính mà không cần sửa đổi toàn bộ hệ thống là một đặc tính quan trọng của hệ thống phân tán.

---

23. GIAO TIẾP GIỮA CÁC THÀNH PHẦN

Đề xuất sử dụng:

```
HTTP REST API
```

cho các thao tác nghiệp vụ.

Sử dụng:

```
WebSocket
```

cho cập nhật realtime đến frontend.

Toàn bộ dữ liệu trao đổi sử dụng:

```
JSON
```

---

24. CÔNG NGHỆ ĐỀ XUẤT

Phần này là **đề xuất triển khai kỹ thuật**, không phải yêu cầu bắt buộc trong PDF môn học.

Backend

Có thể chọn:

```
Node.js + Express
```

Lý do:

- Code nhanh.
- REST API đơn giản.
- Dễ tạo nhiều service.
- Dễ xử lý request bất đồng bộ.

Frontend

```
ReactJS
```

Database

Đề xuất:

```
PostgreSQL
```

hoặc:

```
MySQL
```

Mỗi Parking Node sử dụng database/container riêng.

Realtime

```
Socket.IO / WebSocket
```

Deployment

```
Docker
Docker Compose
```

Docker rất phù hợp cho việc chứng minh:

```
Parking A = process/container riêng
Parking B = process/container riêng
Parking C = process/container riêng
Aggregator = process/container riêng
```

---

25. CẤU TRÚC PROJECT

Đề xuất:

```
smart-parking-distributed/
│
├── frontend/
│
├── aggregator-service/
│
├── parking-node/
│
├── docker/
│
├── docs/
│
├── docker-compose.yml
└── README.md
```

Không cần copy code Parking Node thành ba project hoàn toàn khác nhau.

Có thể dùng chung image:

```
parking-node
```

nhưng chạy 3 instance:

```
parking-node-a
parking-node-b
parking-node-c
```

với environment khác nhau.

---

26. DOCKER COMPOSE DỰ KIẾN

```
frontend

aggregator

parking-a
db-a

parking-b
db-b

parking-c
db-c
```

Sơ đồ:

```
Docker Network

 ├── frontend
 │
 ├── aggregator
 │       ├── parking-a
 │       ├── parking-b
 │       └── parking-c
 │
 ├── parking-a → database-a
 ├── parking-b → database-b
 └── parking-c → database-c
```

---

27. GIAO DIỆN DỰ KIẾN

Trang 1 – Dashboard

Hiển thị:

```
----------------------------------------
 SMART PARKING
----------------------------------------

Parking A
12 / 20 Available
[View]

Parking B
7 / 30 Available
[View]

Parking C
OFFLINE

----------------------------------------
```

Trang 2 – Chi tiết bãi

```
Parking A

A01 AVAILABLE
A02 OCCUPIED
A03 RESERVED
A04 AVAILABLE
A05 MAINTENANCE
```

Sử dụng sơ đồ trực quan để hiển thị slot.

Trang 3 – Đặt chỗ

Người dùng nhập:

- Biển số.
- Thời gian.
- Loại phương tiện.

Sau đó chọn slot.

Trang 4 – Admin Dashboard

Hiển thị:

```
NODE      STATUS      AVAILABLE

A         ONLINE        12
B         ONLINE         7
C         OFFLINE        -
```

---

28. CÁC TRẠNG THÁI SLOT

Đề xuất sử dụng 4 trạng thái:

```
AVAILABLE
```

Có thể đỗ.

```
RESERVED
```

Đã được đặt.

```
OCCUPIED
```

Đang có xe.

```
MAINTENANCE
```

Không được sử dụng.

Luồng trạng thái:

```
AVAILABLE
   ↓
RESERVED
   ↓
OCCUPIED
   ↓
AVAILABLE
```

Hoặc:

```
AVAILABLE
   ↓
OCCUPIED
```

```
   ↓
AVAILABLE
```

nếu xe vào mà không đặt trước.

---

29. XỬ LÝ TƯƠNG TRANH

Giả sử:

```
User A
User B
```

cùng đặt:

```
Slot A05
```

tại cùng thời điểm.

Không được xảy ra:

```
A đặt thành công
B cũng đặt thành công
```

Parking Node phải đảm bảo:

```
chỉ một reservation thành công.
```

Có thể xử lý bằng:

- Database transaction.
- Row locking.
- Atomic update.

Đây là phần nâng cao quan trọng để hệ thống có tính thực tế.

---

30. BẢO MẬT CƠ BẢN

Có thể triển khai:

```
JWT Authentication
```

Phân quyền:

```
USER
STAFF
ADMIN
```

Các API nội bộ giữa Aggregator và Parking Node có thể sử dụng:

```
Internal API Key
```

---

31. KỊCH BẢN DEMO

Khi bảo vệ nên demo theo đúng trình tự.

Demo 1

Chạy:

```
Parking A
Parking B
Parking C
Aggregator
Frontend
```

Frontend hiển thị:

```
A: 10
B: 7
C: 5
```

Demo 2

Tại Parking A cho xe vào.

```
A01
AVAILABLE → OCCUPIED
```

Frontend thay đổi gần như ngay lập tức:

```
A: 10 → 9
```

Demo 3

Người dùng đặt:

```
Parking B – Slot B05
```

Trạng thái:

```
AVAILABLE → RESERVED
```

Demo 4

Tắt:

```
parking-b
```

Aggregator hiển thị:

```
Parking B OFFLINE
```

nhưng:

```
Parking A → hoạt động
Parking C → hoạt động
```

Demo 5

Khởi động Parking B trở lại.

Health check phát hiện:

```
Parking B ONLINE
```

và dữ liệu lại được đưa vào kết quả tổng hợp.

---

32. KIỂM THỬ HỆ THỐNG

TC01 – Tra cứu nhiều node

Input: yêu cầu xem toàn bộ bãi.

Expected:

Aggregator lấy dữ liệu từ A, B, C và tổng hợp chính xác.

TC02 – Đặt slot

Input:

```
A01 AVAILABLE
```

User đặt A01.

Expected:

```
A01 → RESERVED
```

TC03 – Hai người đặt cùng slot

User 1 và User 2 cùng đặt A01.

Expected:

Chỉ một user thành công.

TC04 – Node lỗi

Tắt Parking B.

Expected:

```
B = OFFLINE
```

A và C vẫn hoạt động.

TC05 – Node phục hồi

Bật Parking B.

Expected:

Aggregator tự phát hiện B quay lại.

TC06 – Xe vào

```
AVAILABLE → OCCUPIED
```

Số slot trống giảm.

TC07 – Xe ra

```
OCCUPIED → AVAILABLE
```

Số slot trống tăng.

---

33. ĐỐI CHIẾU VỚI TIÊU CHÍ CHẤP NHẬN ĐỀ TÀI

Tiêu chí 1

Có ít nhất hai thành phần độc lập giao tiếp qua mạng.

Đề tài có:

```
Aggregator
Parking A
Parking B
Parking C
```

Mỗi thành phần chạy process/container riêng.

Giao tiếp:

```
HTTP REST
```

→ ĐẠT

Tiêu chí 2

Có vấn đề dữ liệu phân tán thực sự.

```
DB A → Parking A
DB B → Parking B
DB C → Parking C
```

Không sử dụng một DB chung cho toàn bộ slot.

→ **ĐẠT**

Tiêu chí 3

Có nghiệp vụ rõ ràng.

Các use case:

1. Tra cứu chỗ trống.
2. Đặt chỗ.
3. Xe vào.
4. Xe ra.
5. Hủy đặt chỗ.
6. Node lỗi/phục hồi.

→ **ĐẠT**

---

34. ĐIỂM KHÁC BIỆT VỚI CRUD THÔNG THƯỜNG

Hệ thống không phải:

```
Frontend
   ↓
Backend
   ↓
Database
```

mà là:

```
                Aggregator
               /     |
              /      |
             A       B       C
```

```
        ↓       ↓       ↓
      DB A    DB B    DB C
```

Do đó có:

- Phân tán xử lý.
- Phân tán dữ liệu.
- Giao tiếp mạng.
- Node độc lập.
- Tổng hợp dữ liệu.
- Xử lý lỗi node.
- Cập nhật trạng thái gần thời gian thực.

---

35. LIÊN HỆ VỚI KIẾN THỨC MÔN HỌC

Mô hình Client/Server trong tài liệu được mô tả là hình thức trao đổi thông tin giữa tiến trình cung cấp dịch vụ và tiến trình sử dụng dịch vụ, trong đó máy khách gửi yêu cầu và máy chủ xử lý rồi trả kết quả.

Các tiến trình máy khách và máy chủ có thể được cài đặt trên những máy khác nhau và trao đổi thông tin qua mạng bằng tin báo.

Trong hệ thống đề xuất:

```
Frontend
```

đóng vai trò client đối với Aggregator.

```
Aggregator
```

đóng vai trò server đối với Frontend nhưng đồng thời lại đóng vai trò client khi gọi các Parking Node.

```
Parking Node
```

đóng vai trò server cung cấp dịch vụ quản lý từng bãi.

Điều này phù hợp với nhận xét trong tài liệu rằng khái niệm client/server có tính tương đối: một module có thể đóng vai trò server với module này nhưng là client với module khác.

---

36. KẾ HOẠCH TRIỂN KHAI

Giai đoạn 1 – Xây dựng Parking Node

Thực hiện:

- Database.
- Slot API.
- Reservation API.
- Enter/Exit API.

Kết quả:

```
1 Parking Node hoạt động hoàn chỉnh.
```

Giai đoạn 2 – Nhân thành nhiều node

Chạy:

```
Parking A
Parking B
Parking C
```

Mỗi node:

```
port riêng
database riêng
parkingId riêng
```

Giai đoạn 3 – Aggregator

Xây dựng:

- Node registry.
- Health check.
- Parallel request.
- Aggregated availability.
- Request routing.

Giai đoạn 4 – Frontend

Xây:

- Dashboard.
- Parking detail.
- Slot map.
- Reservation.
- Admin monitoring.

Giai đoạn 5 – Realtime

Thêm:

```
WebSocket
```

Cập nhật xe vào/ra lên frontend.

Giai đoạn 6 – Fault tolerance

Thử:

```
kill Parking B
```

và đảm bảo toàn hệ thống vẫn hoạt động.

Giai đoạn 7 – Docker

Docker hóa toàn bộ:

```
frontend
aggregator
parking nodes
databases
```

Chạy bằng:

```
docker compose up
```

---

37. PHÂN CHIA CÔNG VIỆC NHÓM

37. PHÂN CHIA CÔNG VIỆC NHÓM (BẢN 6 THÀNH VIÊN)
Phân công nhiệm vụ cụ thể cho 6 thành viên theo kế hoạch chi tiết:
• Vũ Văn Hùng: Parking Node + Database (thiết kế CSDL bãi, xử lý tương tranh, chống đặt trùng).
• Hoàng Văn Huynh: Aggregator Service + Điều phối (định tuyến, tra cứu song song, health check, bảo mật).
• Nguyễn Văn Luân: Frontend (Web React + Mobile Responsive, realtime client, barrier demo).
• Trịnh Kim Loan: Outbox + RabbitMQ (Transactional Outbox, luồng sự kiện, chống mất/trùng tin, đối soát).
• Đỗ Huyền Trang: Docker + Triển khai + Test e2e (Docker Compose, kịch bản kiểm thử, kiểm tra bất biến).
• Phạm Sỹ Hiệp: Báo cáo + Slide + Tài liệu HLD (mô hình HLD, phân tích CAP, tài liệu triển khai).

Sau đó tất cả thành viên cùng: Tích hợp hệ thống, kiểm thử liên thông, hoàn thiện báo cáo và tập demo.

38. KẾT QUẢ DỰ KIẾN

Sau khi hoàn thành, hệ thống phải chứng minh được:

1. Có nhiều Parking Node hoạt động độc lập.
2. Mỗi node sở hữu dữ liệu riêng.
3. Các node giao tiếp thông qua mạng.
4. Người dùng có thể tra cứu nhiều bãi từ một giao diện.
5. Aggregator có thể tổng hợp dữ liệu nhiều node.
6. Trạng thái slot được cập nhật khi xe vào/ra.
7. Có thể đặt chỗ.
8. Có cơ chế ngăn đặt trùng slot.
9. Một node bị lỗi không làm toàn bộ hệ thống dừng.
10. Có khả năng thêm Parking Node mới.

---

39. HƯỚNG PHÁT TRIỂN

Sau phiên bản môn học có thể mở rộng:

- Tích hợp camera nhận diện biển số.
- QR Code cho xe vào/ra.
- Cảm biến IoT tại từng vị trí.
- Thanh toán online.
- Bản đồ tìm bãi gần nhất.
- Dự báo số lượng chỗ trống.
- Thống kê giờ cao điểm.

- Giá đỗ xe động.
- Notification.
- Mobile Application.
- Hệ thống message broker.
- Replication và failover nâng cao.

Các chức năng này là **hướng mở rộng**, không nên đưa hết vào phiên bản đầu tiên vì có thể làm phạm vi đề tài quá lớn.

---

40. KẾT LUẬN

Đề tài **Bãi đỗ xe thông minh liên kết nhiều bãi** được triển khai theo kiến trúc hệ thống phân tán với nhiều Parking Node hoạt động độc lập.

Mỗi bãi sở hữu dữ liệu của riêng mình, trong khi Aggregator cung cấp một điểm truy cập thống nhất để người dùng tìm kiếm, tra cứu và đặt chỗ trên toàn hệ thống.

Kiến trúc đề xuất đáp ứng ba yêu cầu cốt lõi của đề tài:

- Có nhiều thành phần độc lập giao tiếp qua mạng.
- Có dữ liệu phân tán thực sự giữa các node.
- Có các nghiệp vụ cụ thể vượt ra ngoài CRUD đơn giản.

Đồng thời, hệ thống thể hiện được các đặc trưng quan trọng của hệ thống phân tán được trình bày trong tài liệu môn học như tính trong suốt, giao tiếp giữa các tiến trình, khả năng mở rộng và tổ chức theo mô hình Client/Server.

Phiên bản triển khai ban đầu nên tập trung vào:

3 Parking Node + 3 Database riêng + Aggregator + Frontend + cập nhật realtime + xử lý node offline.

Đây sẽ là phạm vi vừa đủ để triển khai trong thời gian môn học, đồng thời chứng minh rõ ràng bản chất **hệ thống phân tán** của đề tài.